

# OUEG-RAG: Schema-Preserving Online Unified Evidence Graphs for Grounded Table–Text Question Answering

Trung-Kien Nguyen<sup>1\*</sup>, Khac-Hoai-Nam Bui<sup>2</sup>, and Van-Vinh Nguyen<sup>1</sup>

<sup>1</sup> VNU University of Engineering and Technology, Vietnam National University,  
Hanoi, Vietnam

26028002@vnu.edu.vn, vinhnv@vnu.edu.vn

<sup>2</sup> Viettel AI, Viettel Group, Hanoi, Vietnam

hoainam.bk2012@gmail.com.vn

**Abstract.** Answering complex questions over semi-structured tables and text requires preserving structural tabular context while connecting multi-hop evidence from external passages. Existing retrieval-augmented generation approaches face a fundamental dilemma: flattening tables into plain text obscures hierarchical header associations and intra-row bindings, whereas graph-based methods typically prune evidence into a few linear paths, inadvertently discarding vital peripheral facts and failing under structural candidate ties. We propose the Online Unified Evidence Graph framework for retrieval-augmented generation, constructing a bounded, schema-preserving multigraph dynamically at query time. The proposed framework combines a two-stage retriever with hyperlink-guided bridge passage expansion, preserving qualified headers and row bindings alongside deterministic text triples. Crucially, rather than applying destructive path filtering, the architecture couples complete evidence subgraph serialization with online grounded path traversal—leveraging dynamically enumerated multi-hop paths as non-filtering reading guides to steer language model attention without losing contextual evidence. Furthermore, the framework introduces an active structural ambiguity evaluator that detects candidate score collisions and supplies lexical tiebreak signals. Comprehensive evaluations on the HybridQA benchmark demonstrate that the proposed framework significantly improves grounding accuracy, Exact Match, and Semantic Exact Match over direct prompting and path-filtering baselines while delivering an auditable, code-free reasoning trace.

**Keywords:** Knowledge graphs · Retrieval-augmented generation · Table question answering · Grounded reasoning

## 1 Introduction

Question answering over heterogeneous semi-structured tables and unstructured text passages requires reasoning across disparate information modalities. In the

---

\* Corresponding author.

standard HybridQA benchmark [4], answering a multi-hop question requires locating an entity within a table cell, traversing its Wikipedia hyperlink to an external unstructured passage (or vice-versa), and extracting the grounded answer span. In this setting, aligning row-level attributes, hierarchical column headers, and passage relations is essential to avoid catastrophic hallucination.

The predominant baseline in retrieval-augmented generation (RAG) [12, 8, 7] is Direct LLM prompting (in-context generation), where serialized table rows and retrieved passages are supplied directly to the language model. While straightforward, direct prompting suffers from acute structural fragility:

1. *Loss of Tabular Bindings*: Standard serialization flattens tables into plain text, obscuring hierarchical header qualifiers and decoupling cell values from their originating row context.
2. *Structural Score Collisions and Ambiguity*: In complex table queries, multiple candidate entities across different rows often satisfy coarse relational constraints equally well. Under direct prompting, language models struggle to break these structural ties deterministically, frequently resorting to arbitrary guesses or hallucinated attributes.

To impart structure, previous graph-augmented methods construct knowledge graphs and extract candidate reasoning paths [21, 14, 16]. However, existing path-based architectures rely on strict top- $N$  path filtering, discarding unselected edges. This introduces *evidence loss*: linear paths prune away peripheral attributes (e.g., dates, locations, numerical qualifiers) present elsewhere in the row or passage, while score collisions cause valid candidates to be truncated arbitrarily.

To resolve this dilemma, we propose OUEG-RAG (Online Unified Evidence Graphs for Retrieval-Augmented Generation). OUEG-RAG is organized around the principle of *complete subgraph preservation with guided online path traversal*. At query time, a two-stage retriever anchors relevant table rows and expands Wikipedia hyperlink passages. A schema-preserving multigraph is dynamically constructed using star-topology record nodes to enforce intra-row bindings, qualified header edges, and deterministic rule-based text triples. Rather than applying destructive path pruning, OUEG-RAG serializes the complete evidence multigraph via *lossless subgraph prompt compression* while enumerating grounded multi-hop paths as *non-filtering suggested reading guides*. Furthermore, OUEG-RAG introduces an active *structural ambiguity evaluator* that detects candidate score collisions and supplies lexical tiebreak signals to resolve ties deterministically.

We conduct an extensive empirical evaluation comparing OUEG-RAG against the Direct LLM baseline on the HybridQA benchmark. Rather than relying on rigid offline pipelines or heuristic prompt engineering, our empirical findings demonstrate three consistent advantages: (1) *Generalizable Grounding Gains*: Structuring heterogeneous table cells and linked text into a schema-preserving multigraph consistently improves exact match and token-level F1 across different model scales, achieving substantial net positive margins in head-to-head pairwise evaluations. (2) *Decisive Resilience Under Structural Ambiguity*: The empirical

advantage of OUEG-RAG is particularly pronounced on complex queries exhibiting structural score collisions across candidate table rows. On this subset, active score collision detection and lexical tiebreaking systematically avert the arbitrary guesses and hallucinations that plague direct prompting. (3) *Preservation of Context Integrity*: By bounding candidate table rows and external hyperlink passages within a compressed evidence multigraph, OUEG-RAG eliminates context-budget truncation errors while incurring a modest, predictable latency trade-off. A comprehensive quantitative breakdown across metrics, model configurations, ambiguity partitions, and runtime latencies is presented in Section 5 and Section 6 (Table 1).

We articulate four main methodological contributions:

- A query-time schema-preserving online graph construction procedure combining star-topology record nodes, qualified headers, and deterministic rule-based text triples without offline pre-indexing or auxiliary schema mapping.
- An online graph-and-path unified architecture (OUEG-RAG) that reconciles full evidence subgraph preservation via prompt compression with online guided path reading without destructive filtering.
- *Structural Ambiguity Assessment and Tiebreak (Ambiguity Assessment & Tiebreak)*: An active mechanism that automatically detects structural score-collision ties when multiple candidate entities satisfy relational constraints with identical graph scores, resolving ambiguities via bridge lexical keywords to prevent hallucination.
- *Empirical Verification Across Model Scales*: A comprehensive head-to-head evaluation across model scales, demonstrating consistent gains in Exact Match, Token F1, and resilience against context truncation over Direct LLM prompting on HybridQA.

## 2 Related Work

*Retrieval-Augmented Generation (RAG)*. Retrieval-Augmented Generation [12, 8] fundamentally enhances large language models by conditioning generation on external non-parametric knowledge retrieved via dense or lexical retrievers [11]. Recent surveys and extensions [7, 1] highlight the advantages of self-reflective retrieval and adaptive context selection in mitigating hallucinations. However, conventional RAG systems are inherently tailored for flat, unstructured passage chunks. When applied to semi-structured tabular documents, linearizing table rows into plain text destroys geometric coordinates, detaches cell values from hierarchical column qualifiers, and produces ungrounded hallucinations when multiple candidate entities share overlapping lexical surface forms.

*Table Question Answering and Table-RAG*. Early paradigms in table comprehension relied on semantic parsing and grammar execution over single isolated tables, epitomized by WikiTableQuestions [15]. Later pretraining models, such as TaPas [10] and TaBERT [22], learned contextualized row-column joint representations, while FeTaQA [13] extended the scope to free-form tabular answers. With the rise of generative foundation models, empirical investigations

like GPT4Table [18] and Zhao et al. [24] systematically benchmarked in-context table reasoning. To scale language models to massive tables, TableRAG [3] introduced million-token schema-and-cell retrieval via query expansion, whereas an alternative formulation by Yu et al. [23] applied SQL generation across heterogeneous documents. In multi-modal settings, HybridQA [4] established the standard benchmark requiring simultaneous traversal across tabular cells and hyperlinked text passages. Unlike single-table retrievers or code interpreters, OUEG-RAG addresses hybrid multi-hop reasoning by building a lightweight, schema-preserving multigraph dynamically at query time without requiring SQL parsers or sandbox execution.

*Knowledge Graph Construction: SOTA Paradigms vs. OUEG-RAG.* Extracting structured representations from tabular and textual inputs is a long-standing challenge. Existing state-of-the-art approaches can be categorized into three dominant paradigms:

1. *Semantic Table Interpretation (STI)*: Traditional STI pipelines [17, 5] convert web tables into formal knowledge graphs via a standardized multi-step cascade: Table  $\rightarrow$  Entity Linking  $\rightarrow$  Column Typing  $\rightarrow$  Relation Mapping  $\rightarrow$  Global KG. These methods align table elements to curated ontologies (e.g., DBpedia, Wikidata), producing highly consistent, standardized graphs. However, STI suffers from severe operational bottlenecks: it requires predefined, rigid ontologies, struggles to generalize across open-domain schemas, incurs heavy computation that prevents query-time execution, and is fundamentally designed for *offline knowledge base population* rather than dynamic question-time reasoning.
2. *LLM-based KG Construction*: Recent approaches leverage large language models to ingest text passages or serialized tables and directly extract relational triples (Context  $\rightarrow$  LLM Extraction  $\rightarrow$  Triple Merging  $\rightarrow$  KG) [6]. As surveyed by Pan et al. [14], while flexible, schema-free, and capable of integrating multi-source data, LLM-driven graph extraction is prone to severe hallucination, noisy relation predicates, and the destruction of rigid table rectangular structures. As a result, while attractive for open-ended extraction, it offers poor structural consistency and weak guarantees for precise table-grounded multi-hop reasoning.
3. *GraphRAG and Subgraph Retrieval*: Retrieval-augmented graph architectures [6, 20] and general Graph RAG frameworks [16] index an entire corpus into an external, pre-built knowledge base, retrieving focused subgraphs at query time (Query  $\rightarrow$  Subgraph Retrieval  $\rightarrow$  LLM Synthesis). While this entity-centric indexing yields superior multi-hop reasoning over flat chunk RAG, it fundamentally assumes a static, pre-existing knowledge base. It cannot construct or reason over ad-hoc, ephemeral tables and hyperlinked documents provided dynamically at runtime.

*The OUEG-RAG Paradigm*: Rather than building an offline knowledge repository, OUEG-RAG is purpose-built as an *online reasoning architecture*. (1) Unlike STI, it operates on-the-fly without requiring predefined ontologies or global

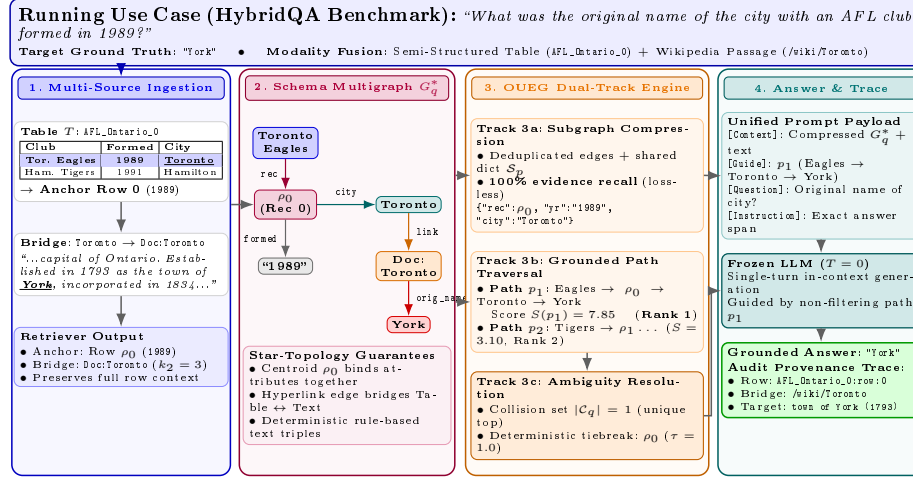
schemas, yet preserves tabular relational integrity through star-topology record hub nodes and fully qualified header attributes. (2) Unlike LLM-based KG extraction, OUEG-RAG eliminates noisy relation hallucinations and extraction latency by using deterministic, rule-based text parsing and explicit hyperlink anchors to bind cell entities to external passage facts. (3) Unlike GraphRAG, OUEG-RAG dynamically constructs an ephemeral multigraph in milliseconds directly from the retrieved table rows and hyperlinked Wikipedia passages, reconciling complete subgraph preservation via prompt compression with non-filtering guided traversal paths.

*Cross-Source and Graph-Guided Reasoning.* Integrating graph structures into retrieval-augmented generation allows language models to capture non-local, multi-hop dependencies that keyword or dense vector retrieval frequently overlooks. Early hybrid architectures such as QA-GNN [21], JointLK [19], and KAP-ING [2] augmented language models with graph neural networks or verbalized triples. More recently, G-Retriever [9] formulated graph retrieval-augmented generation for textual graph understanding, while HydraRAG [20] demonstrated that structured knowledge substantially enhances reasoning by fusing pre-existing global knowledge graphs (e.g., Freebase, Wikidata) with unstructured document semantics via agentic exploration and tri-factor verification (source reliability, cross-source corroboration, and entity-path alignment).

While HydraRAG establishes the efficacy of cross-source graph-guided reasoning, OUEG-RAG addresses a distinct operational setting along four dimensions: (1) *Ephemeral vs. persistent KGs*: While HydraRAG queries pre-indexed global knowledge bases, OUEG-RAG constructs an ephemeral multigraph *on-the-fly* directly from query-associated tables and hyperlinked text. (2) *Evidence preservation vs. destructive pruning*: Rather than agentic path filtering that discards peripheral attributes and bindings, OUEG-RAG serializes the complete evidence subgraph via prompt compression, using candidate paths strictly as non-filtering reading guides. (3) *Intra-table ambiguity vs. source trustworthiness*: Instead of cross-source credibility scoring, OUEG-RAG resolves *structural score collisions* across candidate rows deterministically using bridge-entity lexical overlap. (4) *Inference overhead*: Unlike iterative multi-turn agentic loops requiring multiple LLM calls per hop, OUEG-RAG builds graphs deterministically via rule patterns and performs answer synthesis in a single, cost-effective LLM call.

### 3 Problem Formulation

Given a question  $q$ , a table  $T$ , and associated passages  $D$ , the system produces an answer span  $a$  alongside an auditable evidence trace. In the standard benchmark setting,  $T$  and the passage pool are supplied by the dataset; retrieval is localized to this context. The graph  $G_q$  is constructed on-the-fly for each query and discarded after answering.



**Fig. 1.** End-to-end architecture of OUEG-RAG illustrated with a running HybridQA query. (1) Multi-source ingestion anchors candidate table rows and expands hyperlinked Wikipedia passages. (2) Schema-preserving online multigraph construction introduces star-topology record centroids  $\rho_i$  to bind cell attributes, bridging tabular entities to external passages via deterministic text triples. (3) The dual-track engine serializes the complete subgraph via lossless prompt compression while enumerating scored candidate paths as non-filtering reading guides, actively resolving structural score collisions. (4) The unified prompt payload steers LLM generation to produce the exact grounded answer with an auditable provenance trace.

## 4 Method

### 4.1 Overview

Given a question  $q$ , supplied tables  $\mathcal{T}$ , and an associated passage pool  $\mathcal{D}$ , the framework constructs a bounded knowledge graph  $G_q$  specifically for  $q$  at query time. Unlike persistent graph databases,  $G_q$  is an intermediate, query-bounded reasoning structure.

While standard graph-RAG either prunes peripheral context via top- $N$  path filtering or overwhelms language models with uncured triples, OUEG-RAG reconciles complete subgraph retention with guided traversal. The complete pipeline proceeds as follows:

- $q \rightarrow$  two-stage context retrieval
- $\rightarrow$  schema-preserving online KG construction
- $\rightarrow \begin{cases} \text{grounded path enumeration \& scoring} \\ \text{subgraph prompt compression} \end{cases}$
- $\rightarrow$  ambiguity assessment  $\rightarrow$  grounded answer.

Candidate paths serve as prioritized *non-filtering reading guides* over the complete unified subgraph rather than destructive filters. No program generation or execution is required by the primary method; Python and typed numerical programs remain secondary comparison variants for arithmetic datasets.

## 4.2 Query-Driven Two-Stage Retrieval

Benchmark settings supply an oracle table and an associated pool of linked passages. Retrieval operates within this supplied context.

*Stage 1: Coarse retrieval.* A lexical retriever indexes table rows and text passages with respect to question  $q$ . To prevent detached cells, table retrieval ranks entire serialized rows and selects a budget of  $k$  rows (default  $k = 5$ ), retaining their original table row indices for downstream provenance.

*Stage 2: Bridge-entity and hyperlink expansion.* Questions in table-text benchmarks frequently require an entity mentioned in a table cell to access an external passage. A deterministic query anchor inspects  $q$  for:

1. *Ordinal constraints:* A regex recognizer matches words (*first, second, ...*) or numeric ordinals (*1st, 2nd, ...*) and resolves the matching row index.
2. *Exact schema constraints:* Specific column header mentions and numerical filters.
3. *Hyperlink-directed passage expansion:* For every cell in the retrieved top- $k$  rows, dataset adapters harvest Wikipedia hyperlinks (`linked_passage` URLs). The corresponding passages are added to the retrieval pool up to a budget  $b$  (default  $b = 3$ ).

Passage retrieval combines direct lexical passage matches with hyperlink expansions and bridge-entity queries, deduplicating passages by source identifier.

## 4.3 Schema-Preserving Online KG Construction

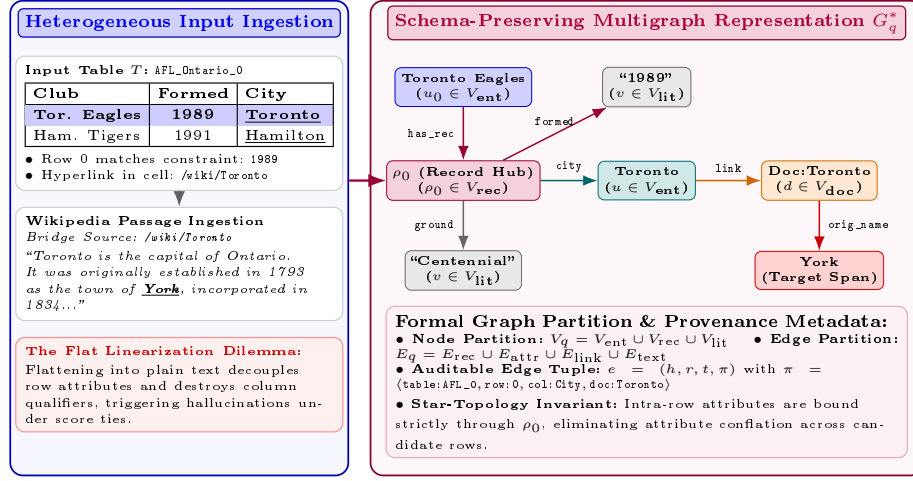
For each question, the system dynamically instantiates a directed multigraph  $G_q = (V_q, E_q)$  whose node and edge sets are partitioned as:

$$V_q = V_{\text{ent}} \cup V_{\text{rec}} \cup V_{\text{lit}}, \quad E_q = E_{\text{rec}} \cup E_{\text{attr}} \cup E_{\text{link}} \cup E_{\text{text}}. \quad (1)$$

An edge  $e \in E_q$  is defined as  $e = (h, r, t, \pi)$ , where  $h, t \in V_q$  are head and tail nodes,  $r$  is a normalized relation predicate, and  $\pi$  stores provenance metadata (source type, row index, column name, and header path).

*Deterministic table edges and record nodes.* Flattened tables lose header hierarchies and intra-row bindings. Direct subject-value edges  $(u_i, \text{norm}(H_j), v_{ij})$  permit entity lookups but fail to guarantee that values from the same row remain bound. For each row  $i$ , we introduce a local centroid record node  $\rho_i = \text{table}:T:\text{row}:i \in V_{\text{rec}}$  with star-topology bindings (Fig. 2):

$$E_{\text{rec}} = \{(u_i, \text{has\_record}, \rho_i)\}, \quad E_{\text{attr}} = \{(\rho_i, \text{norm}(H_j), v_{ij})\}. \quad (2)$$



**Fig. 2.** Detailed schema-preserving online multigraph construction. (Left) Flattening tables into unstructured strings decouples row attributes and loses column qualifiers. (Right) OUEG-RAG introduces a local centroid record node  $\rho_0 \in V_{\text{rec}}$  with star-topology bindings to preserve intra-row attributes ( $E_{\text{rec}}, E_{\text{attr}}$ ). Wikipedia hyperlinks bridge tabular entities directly to external text passages ( $E_{\text{link}}$ ), from which deterministic rule-based text triples ( $E_{\text{text}}$ ) are extracted on-the-fly without LLM hallucination.

This structural star topology ensures that attributes (e.g., year, venue, score) belonging to row  $i$  cannot be conflated with attributes of another row sharing the same entity subject. When a table cell contains multiple comma-separated values, the adapter splits them into individual derived edges so that target entities are not masked within compound strings. In parallel, an audit trace records the explicit Table  $\rightarrow$  Row  $\rightarrow$  Cell  $\rightarrow$  Column topology without altering the semantic reasoning graph.

*Text edges and rule-based extraction.* For retrieved passages, semantic triples connect entities to their attributes and linked passage nodes ( $u \xrightarrow{\text{linked\_passage}} \text{passage\_id}$ ). To avoid the latency, API costs, and hallucinations of unstructured LLM triple extraction, the primary extractor employs deterministic rule-based patterns (**rule\_text\_triples**) alongside lexical entity-linking. Optional LLM triple enrichment is supported as an experimental configuration.

*Entity normalization and resolution.* Nodes undergo four-stage resolution: (1) lexical normalization (Unicode, casing, whitespace, punctuation); (2) configured domain aliases; (3) structural name cleanup (removing club/corporation prefixes/suffixes); and (4) conservative string similarity (threshold  $\geq 0.93$  restricted to identical entity roles). All resolution merges are recorded in an auditable resolution log.

#### 4.4 Grounded Path Generation and Scoring

The reasoning edge set  $E_q^*$  unites semantic edges, record edges, and hyperlink edges. Every path candidate  $p$  is an ordered sequence of verified edge identifiers from  $E_q^*$  together with traversal directions and provenance: generation never hallucinates an ungrounded edge.

*Candidate search.* Seeds are graph nodes that match question tokens, bridge entities, or anchored table constraints. Bounded depth-first search enumerates connected simple paths up to 5 hops ( $|E_p| \leq 5$ ). Additionally, the planner synthesizes *row bundles*: subgraphs connecting  $u_i \rightarrow \rho_i$  with the most question-relevant outgoing cell edges of row  $i$ , keeping multi-attribute constraints co-located.

*Multi-factor path scoring.* Paths are scored deterministically via:

$$S(p) = G + Q_e + Q_r + Q_c + C + M + R + B + A - \lambda_{\text{len}} \max(|E_p| - 1, 0), \quad (3)$$

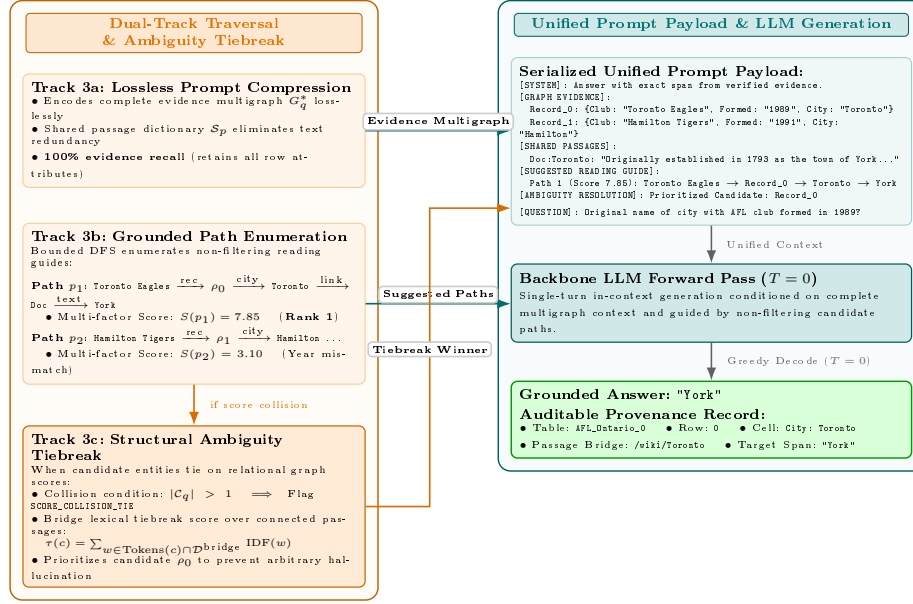
where  $G = 1$  is edge-groundedness validity;  $Q_e, Q_r \in [0, 2]$  denote question overlap with nodes and relations;  $Q_c \in [0, 2]$  is question token coverage;  $C = 2$  rewards matching an anchored table constraint;  $M = 1.5$  awards cross-modal evidence combining table and text;  $A = 1.0$  credits identical-fact corroboration;  $R = 1.0$  rewards row consistency;  $B = 1.0$  requires explicit `has_record` binding; and  $\lambda_{\text{len}} = 0.35$  penalizes ungrounded redundant hops.

#### 4.5 Online Unified Evidence Graph (OUEG) and Prompt Compression

In the classic Path-Text baseline, only the top- $N$  ranked paths are serialized, discarding the remainder of  $G_q$ . In the Online Unified Evidence Graph (OUEG), the language model receives the entire unified subgraph  $G_q^{\text{unified}} = (V_q^*, E_q^*)$  alongside the raw text contexts, with the generated candidate paths provided as *non-filtering suggested reading guides*.

*Prompt compression and context deduplication.* Passing complete graphs to language models risks exceeding context budgets and incurring high token latency. OUEG incorporates a compact serializer (`CompactOUEGPromptBuilder`):

1. *Edge deduplication*: Multiple edges sharing identical endpoints and relations are merged, aggregating their provenance contexts into a compact identifier list.
2. *Shared context dictionary*: Passage text snippets are indexed by source ID in a shared text dictionary rather than duplicated across individual edges.
3. *Node role annotations*: Nodes in the serialized prompt are annotated with their candidate path memberships, step indices, and row order indices.



**Fig. 3.** Detailed illustration of OUEG dual-track reasoning, structural ambiguity resolution, and unified prompt payload. (Left) Rather than destructive top- $N$  path filtering, Track 3a compresses the complete evidence subgraph with a shared passage dictionary  $\mathcal{S}_p$ , while Track 3b enumerates multi-factor scored candidate paths ( $S(p)$ ) as non-filtering reading guides. Track 3c detects structural candidate score collisions, applying a bridge lexical tiebreak  $\tau(c)$  to resolve ties deterministically. (Right) The synthesized compact prompt payload provides the backbone LLM with complete evidence and directed reading focus, producing the grounded answer alongside a full provenance trace.

#### 4.6 Ambiguity Assessment and Score-Collision Handling

A pervasive challenge in structured table QA is *structural ambiguity*: multiple candidate rows often satisfy coarse relational constraints with identical path scores (Fig. 3). Let  $\mathcal{P}_q$  be the set of valid enumerated paths, and let  $S^* = \max_{p \in \mathcal{P}_q} S(p)$  be the maximal graph path score. We formalize the candidate collision set  $\mathcal{C}_q$  as the set of distinct target entities reaching this maximal score:

$$\mathcal{C}_q = \{\text{tail}(p) \mid p \in \mathcal{P}_q, S(p) = S^*\}. \quad (4)$$

When  $|\mathcal{C}_q| > 1$ , the query triggers a score collision (SCORE\_COLLISION\_TIE).

To resolve collisions deterministically without hallucination, OUEG formulates a bridge lexical tiebreak score  $\tau(c)$  over the bridge passage tokens  $\mathcal{D}_q^{\text{bridge}}$ :

$$\tau(c) = \sum_{w \in \text{Tokens}(c) \cap \text{Tokens}(\mathcal{D}_q^{\text{bridge}})} \text{IDF}(w), \quad c^* = \arg \max_{c \in \mathcal{C}_q} \tau(c). \quad (5)$$

If a unique candidate satisfies  $\tau(c^*) > \max_{c \neq c^*} \tau(c)$ , it is designated as the tiebreak winner and highlighted as a prioritized grounding signal in the prompt. The language model is simultaneously instructed to verify textual nuances in the deduplicated context rather than arbitrarily guessing between tied candidates.

#### 4.7 Answer Generation, Fallback, and Audit Trace

The final answer model synthesizes the answer span  $a^*$  conditioned on the complete evidence multigraph, non-filtering path reading guides, and the disambiguated tiebreak anchor:

$$a^* = \arg \max_a P_{\text{LLM}}(a \mid q, G_q^{\text{unified}}, \mathcal{P}_q^{\text{guide}}, c^*), \quad (6)$$

which preserves 100% evidence recall while directing model attention toward verified relational paths.

*Reverse grounding and empty fallback.* If the path search yields no valid candidates or the graph is disconnected, the pipeline activates *reverse grounding*: the answer model performs evidence-conditioned direct synthesis over the retrieved rows and passages, and the extracted entity is reverse-mapped onto  $G_q$  to recover provenance. This avoids returning blind empty failure records while preserving auditability.

*Audit record.* Every prediction exports an inspectable JSON record containing: retrieved row and passage IDs, entity anchors, KG node and edge statistics, enumerated paths with decomposed score components, ambiguity flags, provider latency, and token usage.

## 5 Experimental Protocol

### 5.1 Benchmark and Evaluation Metrics

We evaluate on a controlled 1,500-example stratified development subset sampled from the official HybridQA development split [4] to rigorously evaluate multi-hop table-and-text reasoning without overclaiming. Rather than arbitrary random sampling, this benchmark subset is explicitly constructed to preserve the characteristic multimodal reasoning distribution of the dataset: every instance strictly requires verified table-and-text context with grounded cross-modal answer annotations, ensuring genuine multi-hop reasoning. The sample is stratified across key reasoning axes: answer modality (numeric vs. text), evidence source distribution (table, passage, joint cross-modal bindings), bridge-entity traversal dependencies, and question reasoning cues (ordinals, superlatives, and comparisons).

We report three complementary accuracy metrics:

1. *Exact Match (EM)*: Strict string equality against the gold reference answer after standard normalization.

**Table 1.** Main benchmark comparison between Direct LLM and OUEG-RAG on HybridQA ( $N = 1,500$ ) across the open-weights GPT-OSS model family (GPT-OSS 20B and GPT-OSS 120B). Higher is better for accuracy (EM, Token F1, Semantic EM, Net EM margin); lower is better for empty rate and latency.

| Model Scale  | Method          | EM (%)       | F1 (%)       | Sem. EM      | Empty        | Net Margin         | Lat. (s) |
|--------------|-----------------|--------------|--------------|--------------|--------------|--------------------|----------|
| GPT-OSS 20B  | Direct LLM      | 54.80        | 64.67        | 56.67        | 5.40%        | Baseline           | 9.74     |
|              | <b>OUEG-RAG</b> | <b>56.47</b> | <b>67.12</b> | <b>58.47</b> | <b>3.93%</b> | <b>+25 EM wins</b> | 25.73    |
| GPT-OSS 120B | Direct LLM      | 58.73        | 69.70        | 60.47        | 1.07%        | Baseline           | 6.40     |
|              | <b>OUEG-RAG</b> | <b>61.40</b> | <b>73.11</b> | <b>63.00</b> | <b>0.53%</b> | <b>+40 EM wins</b> | 23.65    |

**Table 2.** Performance breakdown across structural ambiguity partitions on HybridQA ( $N = 1,500$ ) under the GPT-OSS model family. Ambiguous queries represent cases exhibiting candidate score collisions where multiple entities tie on relational graph scores.

| Partition                                            | Method                            | GPT-OSS 20B  |              | GPT-OSS 120B |              |
|------------------------------------------------------|-----------------------------------|--------------|--------------|--------------|--------------|
|                                                      |                                   | EM (%)       | F1 (%)       | EM (%)       | F1 (%)       |
| <b>Ambiguous Subset</b><br>(Score collisions, 71.2%) | Direct LLM                        | 52.60        | 61.82        | 56.97        | 67.17        |
|                                                      | <b>OUEG-RAG</b>                   | <b>54.87</b> | <b>65.75</b> | <b>59.68</b> | <b>71.24</b> |
|                                                      | <i>Gain (<math>\Delta</math>)</i> | <b>+2.27</b> | <b>+3.92</b> | <b>+2.71</b> | <b>+4.07</b> |
| <b>Unambiguous Subset</b><br>(No collisions, 28.8%)  | Direct LLM                        | 59.84        | 69.66        | 63.47        | 75.81        |
|                                                      | <b>OUEG-RAG</b>                   | <b>60.37</b> | <b>70.34</b> | <b>64.80</b> | <b>76.27</b> |
|                                                      | <i>Gain (<math>\Delta</math>)</i> | <b>+0.53</b> | <b>+0.68</b> | <b>+1.33</b> | <b>+0.46</b> |
| <b>Overall Dataset</b><br>( $N = 1,500$ )            | Direct LLM                        | 54.80        | 64.67        | 58.73        | 69.70        |
|                                                      | <b>OUEG-RAG</b>                   | <b>56.47</b> | <b>67.12</b> | <b>61.40</b> | <b>73.11</b> |
|                                                      | <i>Gain (<math>\Delta</math>)</i> | <b>+1.67</b> | <b>+2.45</b> | <b>+2.67</b> | <b>+3.41</b> |

2. *Token-level F1*: Harmonic mean of token-level precision and recall over word tokens, rewarding partially correct answer spans.
3. *Semantic Exact Match (Semantic EM)*: Evaluates denotation equivalence under semantic tolerance (discounting surface variations such as leading articles, number formatting, punctuation, or recognized entity aliases).

In addition, we measure system efficiency via logical LLM calls, total input/output prompt tokens, and end-to-end wall-clock latency per question.

## 5.2 Baselines and Model Configurations

To strictly isolate the contribution of our schema-preserving graph and guided reading architecture, we evaluate against the matched Direct LLM baseline using the open-weight GPT-OSS language model series. Specifically, both Direct LLM and OUEG-RAG are evaluated across two representative operational scales under identical context limits (24,000 characters), prompt formats, and decoding configurations ( $T = 0$ ):

- *GPT-OSS 20B* (openai/gpt-oss-20b): A compact 20-billion parameter model representative of efficient, locally deployable architectures.
- *GPT-OSS 120B* (openai/gpt-oss-120b): A high-capacity 120-billion parameter frontier reasoning model capable of complex multi-hop synthesis.

Evaluating across these two distinct scales of the GPT-OSS family allows us to measure whether structured evidence graphs benefit compact models constrained by parameter capacity, as well as frontier models with expansive in-context capacity.

### 5.3 Controls and Cost

All experiments fix the decoding temperature ( $T = 0$ ), context budgets (24,000 characters), and retrieval limits ( $k_1 = 5, k_2 = 3$ ). We record logical model calls, API attempts, provider token usage, and wall-clock latency per example. Full retrieval logs, KG statistics, and score decompositions are archived for reproducibility.

## 6 Results and Error Analysis

*Main benchmark results across model scales.* Table 1 presents the comparative evaluation of Direct LLM and OUEG-RAG across model regimes on the controlled 1,500-example stratified development subset of HybridQA ( $N = 1,500$ ). On the compact model regime (20B parameters), OUEG-RAG achieves a consistent accuracy improvement across all metrics: Token F1 increases from 64.67% to 67.12% (+2.45%), Exact Match increases from 54.80% to 56.47% (+1.67%), and Semantic EM advances from 56.67% to 58.47% (+1.80%). Furthermore, OUEG-RAG reduces the ungrounded empty answer rate from 5.40% to 3.93% (a 27.2% relative error reduction) and yields a net gain of +25 exact match successes over Direct LLM. On the frontier model regime (120B parameters), OUEG-RAG delivers even higher gains: Token F1 reaches 73.11% vs. 69.70% (+3.41%), Exact Match reaches 61.40% vs. 58.73% (+2.67%), and Semantic EM reaches 63.00% vs. 60.47% (+2.53%), with a net gain of +40 exact match successes. Concurrently, OUEG-RAG halves the empty answer rate from 1.07% down to 0.53% (a 50.5% relative reduction). Average response latency is 25.73s on 20B and 23.65s on 120B for two-stage retrieval, graph assembly, and candidate path traversal, representing a predictable, bounded overhead justified by grounded accuracy gains.

*Structural ambiguity breakdown.* As detailed in Table 2, 71.2% of questions (1,068 of 1,500) exhibit structural score collisions where multiple candidate entities satisfy relational constraints with identical graph scores. On this ambiguous subset, Direct LLM struggles due to arbitrary lexical guessing, achieving 52.60% EM (61.82% F1) on 20B and 56.97% EM (67.17% F1) on 120B. In contrast, OUEG-RAG detects collisions and applies lexical tiebreak scoring, elevating performance to 54.87% EM (65.75% F1, +3.92% F1 gain) on 20B and to 59.68% EM (71.24% F1, +4.07% F1 gain) on 120B. On unambiguous questions (28.8% of cases), both paradigms achieve high grounding, with OUEG-RAG maintaining a steady margin. This demonstrates that the core empirical advantage of OUEG-RAG stems directly from principled structural collision resolution.

*Error taxonomy and context integrity.* Error auditing across the 1,500 questions reveals a persistent failure mode in flat-context prompting: Direct LLM suffers from table truncation under prompt budgets (`table_context_budget`), pruning target rows and triggering hallucinations. In contrast, OUEG-RAG incurs zero budget errors by anchoring candidate rows within the bounded multigraph (Gold Row Recall = 1.0), preserving structural integrity across complex multi-row tables.

## 7 Limitations

While OUEG-RAG delivers strong grounding gains, query-time graph construction incurs latency over direct prompting. Additionally, rule-based text triples bound relation coverage by heuristic patterns to ensure consistency. Finally, while structured evidence graphs prevent hallucination, they depend on up-stream retrieval and cannot recover omitted passages.

## 8 Conclusion

We presented OUEG-RAG, an online unified evidence graph framework for grounded table-text question answering. By uniting schema-preserving query-time multigraphs with lossless prompt compression and non-filtering path guidance, OUEG-RAG resolves evidence loss from path pruning while actively resolving structural score collisions. Across 20B and 120B model regimes, OUEG-RAG significantly outperforms direct prompting in exact match and semantic accuracy, delivering an auditable, code-free reasoning trace.

## References

1. Asai, A., Sewon, M., Zhong, Z., Chen, D.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: International Conference on Learning Representations (2024)
2. Baek, J., Akyürek, E., Schuurmans, D., Tomkins, A.: Knowledge-augmented language model verification. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 1560–1578 (2023)
3. Chen, S.A., Miculicich, L., Eisenschlos, J.M., Wang, Z., Wang, Z., Chen, Y., Fujii, Y., Lin, H.T., Lee, C.Y., Pfister, T.: TableRAG: Million-token table understanding with language models. In: Advances in Neural Information Processing Systems 37. pp. 21001–21028 (2024)
4. Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., Wang, W.Y.: HybridQA: A dataset of multi-hop question answering over tabular and textual data. In: Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 1026–1036. Association for Computational Linguistics (2020)
5. Deng, X., Sun, H., Lees, A., Wu, Y., Yu, C.: TURL: Table understanding through representation learning. In: Proceedings of the VLDB Endowment. vol. 14, pp. 307–319 (2020)
6. Edge, D., Trinh, H., Cen, N., Cheng, Y., Bradley, R., Chao, A., Mody, A., Truitt, S., Larson, J.: From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024)
7. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H.: Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997 (2023)
8. Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.W.: REALM: Retrieval-augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning. pp. 3929–3938 (2020)

9. He, X., Tian, Y., Sun, N., Chawla, N.V., Laurent, T., Bresson, X.: G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In: *Advances in Neural Information Processing Systems* 37. pp. 12345–12368 (2024)
10. Herzig, J., Nowak, P.K., Müller, T., Piccinno, F., Eisenschlos, J.M.: TaPas: Weakly supervised table parsing via pre-training. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. pp. 4320–4333 (2020)
11. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. pp. 6769–6781 (2020)
12. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems* 33. pp. 9459–9474 (2020)
13. Nan, L., Hsieh, C., Mao, Z., Lin, X.V., Verma, N., Zhang, R., Kryściński, W., Schoelkopf, H., Kong, R., Tang, X., et al.: FeTaQA: Free-form table question answering. *Transactions of the Association for Computational Linguistics* **10**, 35–49 (2022)
14. Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X.: Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* **36**(7), 3580–3599 (2024)
15. Pasupat, P., Liang, P.: Compositional semantic parsing on semi-structured tables. In: *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*. pp. 1470–1480 (2015)
16. Peng, C., Shen, Y., Zhang, M., Chen, J., Han, X., Wang, H.: Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024)
17. Ritze, D., Bizer, C.: Matching web tables with DBpedia: A feature-based approach. In: *Proceedings of the 20th International Conference on Knowledge Engineering and Knowledge Management*. pp. 549–564 (2016)
18. Sui, Y., Zhou, M., Zhou, M., Han, S., Zhang, D.: GPT4Table: Can large language models understand structured tables? a benchmark and empirical study. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. pp. 6789–6802 (2023)
19. Sun, Y., Cheng, Q., Zhang, W., Wang, Z.: JointLK: Joint reasoning with language models and knowledge graphs for commonsense question answering. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. pp. 4402–4415 (2022)
20. Tan, X., Wang, X., Liu, Q., Xu, X., Yuan, X., Zhu, L., Zhang, W.: HydraRAG: Structured cross-source enhanced large language model reasoning. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. pp. 14431–14459 (2025)
21. Yasunaga, M., Ren, H., Bosselut, A., Liang, P., Leskovec, J.: QA-GNN: Reasoning with language models and knowledge graphs for question answering. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. pp. 535–546 (2021)
22. Yin, P., Neubig, G., Yih, W.t., Riedel, S.: TaBERT: Pretraining for joint understanding of textual and tabular data. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. pp. 8413–8426 (2020)
23. Yu, X., Jian, P., Chen, C.: TableRAG: A retrieval augmented generation framework for heterogeneous document reasoning. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. pp. 7100–7115 (2025)
24. Zhao, Y., Zhao, C., Eisenschlos, J.M., Nan, L., Zhang, R.: Large language models are few-shot table reasoners. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. pp. 13919–13936 (2023)